

高等電腦視覺

作業#(4)

姓名：闕楷宸

學號：114318047

指導老師：黃正民

作業說明:

作業架構:

主程式HW4_opencv.cpp

輸入0~2即可輸出結果

建置執行(window,linux)

Linux:

- 1.使用opencv 4.5.4(sudo apt install libopencv-dev)
- 2.建置與執行:

```
cd HW#4_114318047/code/  
cmake -S . -B build  
cmake --build build -j  
./build/HW4_opencv
```

Window:

```
cd window_user  
./HW4_opencv.exe
```

Window 建置:

1:Cmake:

```
winget install Kitware.CMake
```

2:OpenCV for window:

```
C:\opencv\build\x64\vc16\bin(make sure DLLs are in this dir)
```

3:Confirm CMake config file exists:

```
C:\opencv\build\x64\vc16\lib\OpenCVConfig.cmake
```

4:Build and run

```
cd HW4_114318047\code
```

5:Configure:

```
cmake -S . -B build -G "Visual Studio 17 2022" -A x64  
-DOpenCV_DIR="C:\opencv\build\x64\vc16\lib"
```

6:Produce exe:

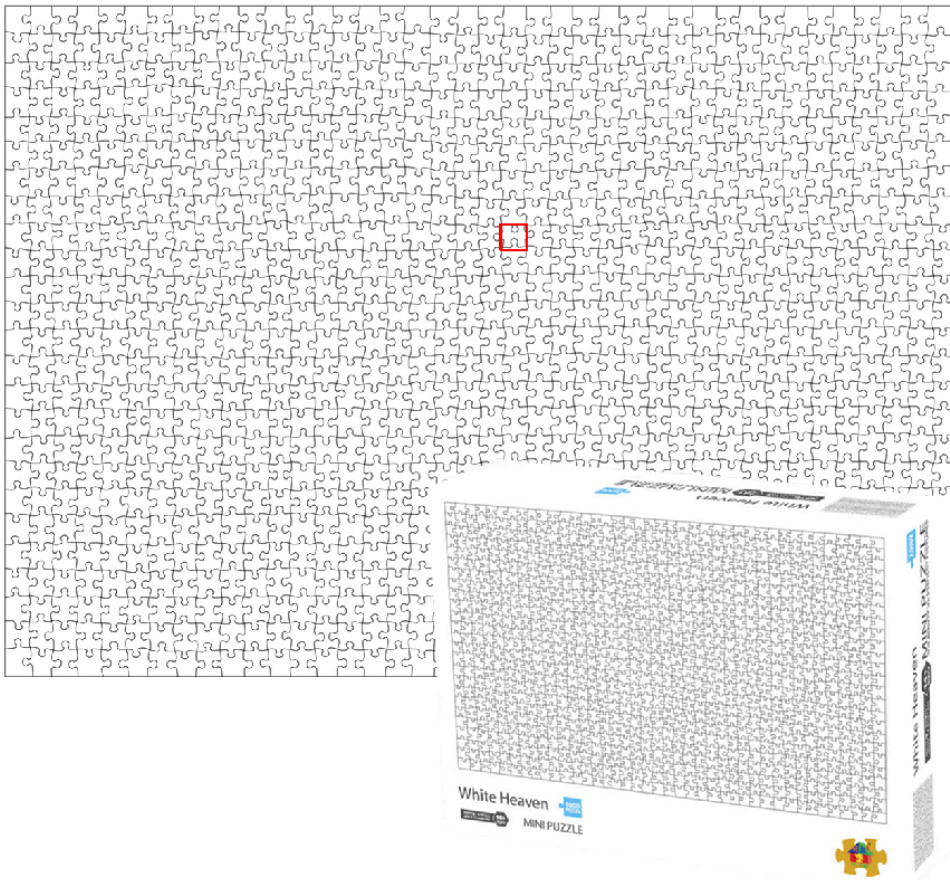
```
cmake --build .\build --config Release
```

7:Run exe

```
.\build\Release\HW4_opencv.exe
```

1. Solve jigsaw

Use template matching with given template to find the piece of jigsaw puzzle



纯白地狱拼图1000片 拼图尺寸：420*297MM 包装尺寸：180*130*50MM

task1_opencv.bmp

Discussion(1)

此題使用OpenCV template matching進行比較，因為模板與拼圖影像尺寸相同，因此只會有一個配對，如task1_opencv.bmp所示，使用function為cv::matchTemplate計算相似後，再使用cv::minMaxLoc找出最大相似度的位置，即可定位並繪製 bounding box。

詳細流程為(1)將彩色轉成灰階影像，專注在邊緣與形狀方面(2)使用cv::matchTemplate時使用TM_CCOEFF_NORMED作為相似度量測方法，降低亮度對結果的影響

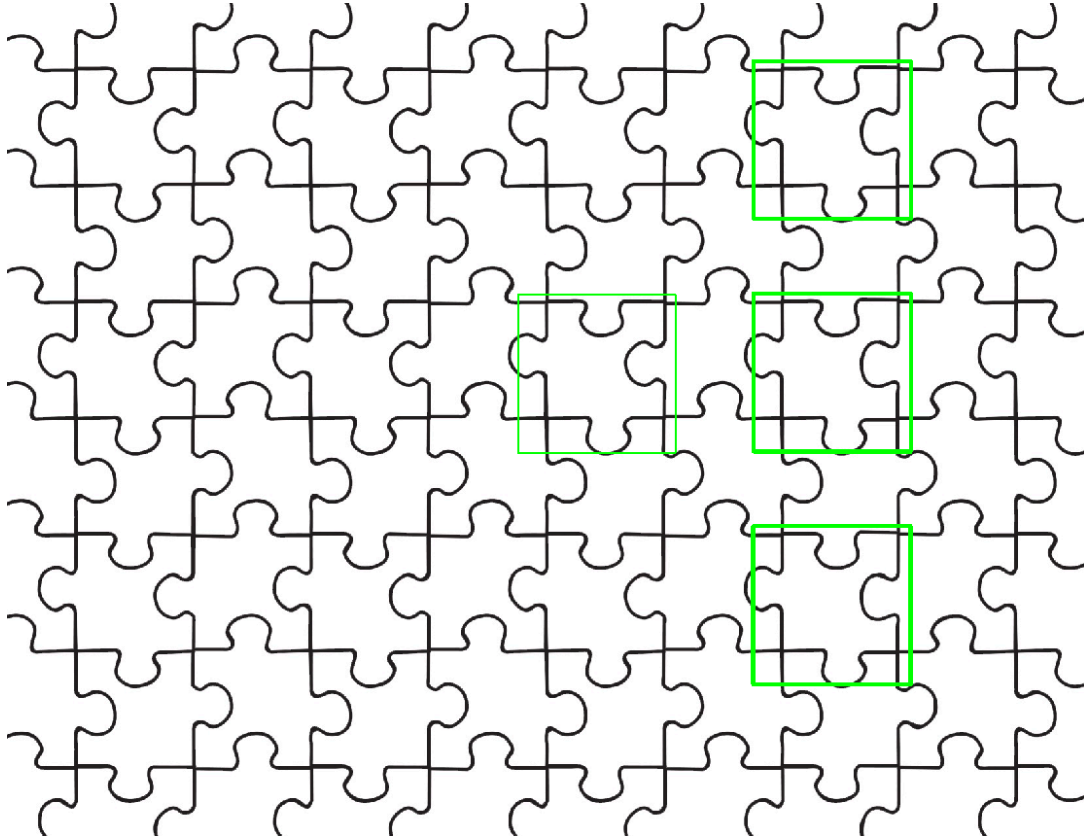
TM_CCOEFF_NORMED:OpenCV會使用template在puzzle上滑動，每個位置都會計算出一個分數並存進result

```
cv::matchTemplate(graySrc, grayTpl, result, cv::TM_CCOEFF_NORMED);
```

並且TM_CCOEFF_NORMED分數會落在[-1,1]，越靠近1(越相似)，0(不相關)，-1(相反)，而在cv::useTemplate後，會使用cv::minMaxLoc找出最相似的配對，使用cv::minMaxLoc則是找尋result中的最高分maxVal，以及最高分位置maxLoc，並交給cv::rectangle(src,maxLoc,cv::Point(maxLoc.x + tpl.cols, maxLoc.y + tpl.rows),cv::Scalar(0, 0, 255),2) 繪製 bounding box。

2.Find the pieces of a jigsaw puzzle

Use template matching with given template to find the pieces.
The scale of the template piece is different with the that of the pieces in the original jigsaw puzzle.



task2_opencv.bmp

Discussion (2)

此題與先前作法類似，但是由於此題的template是可以進行縮放的，因此必須要進行多次template縮放再進行比較，詳細作法如下：

(1) 將template縮放成不同大小再進行比較 (從0.5倍到1.5倍，每次增加0.1)

```
for (double scale = 0.5; scale <= 1.5; scale += 0.1) {
```

(2) 避免縮放的template不會比原圖大

```
if (resizedTpl.cols >= graySrc.cols ||  
    resizedTpl.rows >= graySrc.rows)  
    continue;
```

(3) 計算相似度 (如task1)

```
cv::Mat result;  
cv::matchTemplate(graySrc, resizedTpl, result,  
cv::TM_CCoeff_NORMED);
```

(4) 找出最佳分數

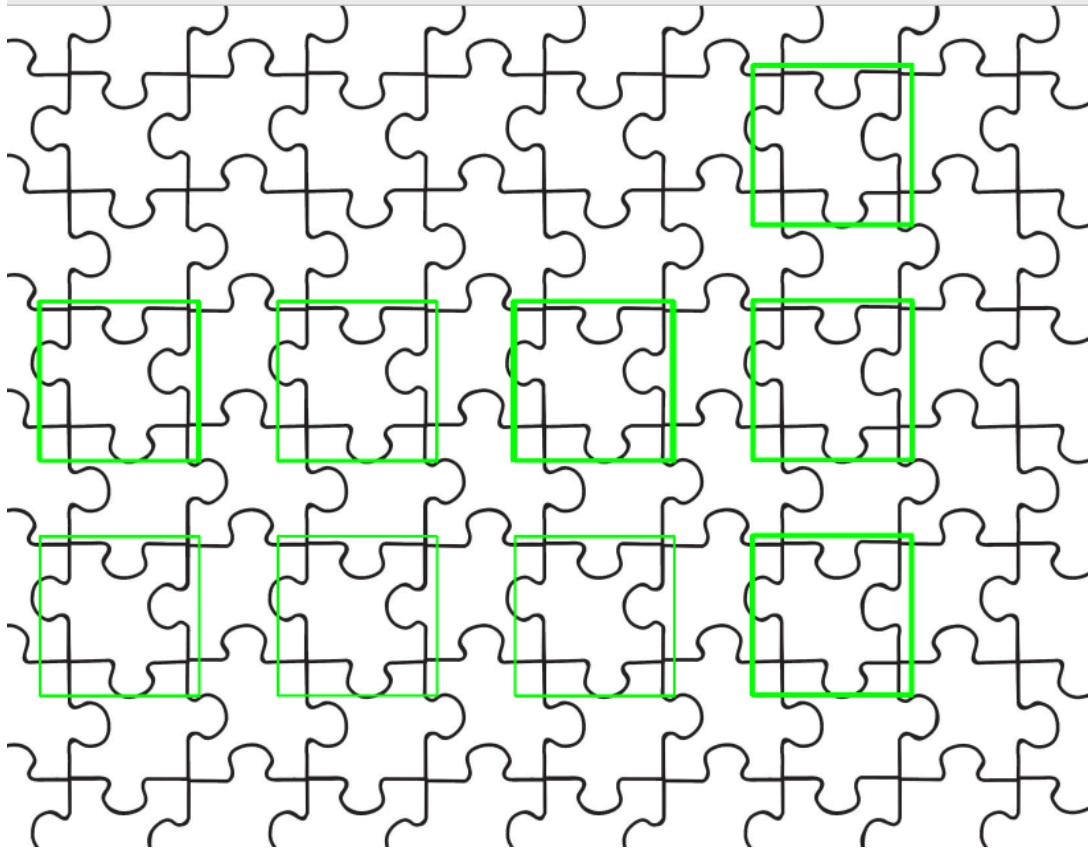
```
double minVal, maxVal;  
cv::Point minLoc, maxLoc;  
cv::minMaxLoc(result, &minVal, &maxVal, &minLoc, &maxLoc);
```

(5) 印出相似分數

```
std::cout << "scale = " << scale  
          << "   maxVal = " << maxVal << std::endl;
```

(6) 比較Threshold

這是比較重要的部份，由於本題的template可以進行縮放，因此會有多個結果輸出，而配對數量會與threshold(相似分數)有關，以下為Threshold 0.4之影像



而以下為0.42之值，可以看出被框出的數量有減少，而有些Bbox的線較粗，是因為重複被匡造成的

